DISEÑO DIGITAL 1.

TÍTULO DE LA ACTIVIDAD: **Ejercicios de modelado de contadores**	**CÓDIGO:** **ATGP3**

FECHA:			
NOMBRE:		APELLIDOS:	

MODALIDAD:	Aprendizaje cooperativo. Ejercicios.	TIPO:	Presencial	DURACIÓN:	50 minutos

CALENDARIO:	Sesión presencial S10.	REQUISITOS:	Conocimientos adquiridos en Electrónica 2.

CRITERIO DE ÉXITO:	

COMENTARIOS E INCIDENCIAS:

TIEMPO DEDICADO:	minutos	AUTOEVALUACIÓN: [entre 0 y 10 puntos]	No procede

Introducción

En esta actividad se incluyen dos ejercicios de modelado de contadores. Para la realización de uno de estos ejercicios es necesario utilizar operaciones de comparación de magnitud, por lo que se presentan las disponibles en el paquete *std_logic_unsigned*. La dinámica del trabajo en grupo será la misma para las partes I y II:

- Inicialmente el G4 se dividirá en 2 parejas. Cada pareja realizará la lectura y el ejercicio.
- Si es necesario, una pareja puede consultar con la otra pareja. Si hay algún problema que no puede resolver el G4, se acudirá al profesor.
- Una vez terminado el ejercicio, se reunirá el G4 y se realizará una puesta en común (si no se ha hecho ya).
- Si hay dudas tras la puesta en común, se consultará al profesor. Si no hay dudas se avisará al profesor de que se ha terminado una parte y se comenzará con la siguiente.

PARTE I. Modelado de contadores con habilitación

Realice las siguientes operaciones:

1. Descargue de Moodle los ficheros correspondientes a esta actividad.

2. Cree un proyecto en el directorio ModelSim de la actividad, denomínelo *Proy_ATGP3* y añádale todos los ficheros del directorio *hdl*.

3. Abra en edición el fichero *cont4b_en.vhd*.

   El código del fichero modela un contador binario ascendente de 4 bits, con entrada de habilitación activa a nivel alto y entrada de reset asíncrono activa a nivel bajo. En este código se pueden apreciar los elementos novedosos, presentados en la actividad anterior, que se utilizan para modelar contadores:

   a. Una cláusula de visibilidad, adicional a las que ya se venían usando, que permite utilizar las operaciones aritméticas para objetos de tipo *std_logic_vector*.

```
-- Cabecera: cláusulas de visibilidad (incluye unsigned)
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
```

   b. Una sentencia de asignación en la que se emplea la operación suma entre un objeto de tipo *std_logic_vector* y una constante entera.

```
-- Proceso que modela el funcionamiento del contador
process(clk, nRST)
begin
  if nRST = '0' then
    Q <= (others => '0');

  elsif clk'event and clk = '1' then
    if ena = '1' then
      Q <= Q + 1;

    end if;
  end if;
end process;
```

Como puede observarse en el código del proceso que modela el funcionamiento (en la sección de código que especifica el funcionamiento síncrono), para especificar que la operación de cuenta está supeditada a la habilitación del contador, la sentencia de asignación que incrementa la cuenta sólo se ejecuta cuando la entrada síncrona *ena* está activa (a nivel alto).

El TB del contador se encuentra en el fichero *cont4b_en_tb.vhd*.

Realice las siguientes operaciones:

1. Abra en edición el fichero *cont4b_en_tb.vhd*.

El test consta de las siguientes pruebas:

a) Prueba de actividad e inactividad de la entrada de inicialización asíncrona.

b) Prueba de cuenta cuando el contador está habilitado.

c) Prueba de mantenimiento del estado de cuenta cuando está inhabilitado.

d) Prueba de la secuencia completa de estados de cuenta.

El código del proceso que codifica estas pruebas se muestra en la figura 1.

```
process
begin
   nRST <= '0';      -- Inicialización asíncrona

  wait until clk'event and clk = '1';
   ena <= '0';
  wait until clk'event and clk = '1';
   nRST <= '1';      -- Desactivación de la inicialización asíncrona
   ena  <= '1';      -- Activación de la habilitación

  wait for 8*T_CLK; -- Se mantiene habilitado el contador 8 ciclos
  wait until clk'event and clk = '1';
   ena <= '0';       -- Inhabilitación

  wait for 2*T_CLK; -- Se mantiene inhabilitado el contador 2 ciclos
  wait until clk'event and clk = '1';
   ena <= '1';       -- Habilitación

  wait for 10*T_CLK; --Se permite la cuenta duarante 10 ciclos de reloj

  wait;              -- Fin del test
end process;

end Test;
```

Figura 1.- Pruebas de test del contador con habilitación

Observe que las pruebas se secuencian alineando los cambios de estado de los estímulos con los flancos de reloj utilizando sentencias WAIT UNTIL para que dichos cambios se produzcan un instante después de la ocurrencia del flanco; cuando un estado de los estímulos se mantiene durante varios ciclos de reloj se recurre a la secuenciación de una sentencia WAIT FOR y una WAIT UNTIL.

Realice las siguientes operaciones:

1. Compile los ficheros con el modelo y TB del contador.
2. Realice una simulación de 2.2 microsegundos para comprobar el funcionamiento del modelo.

Ejercicio

Reutilice el código del modelo del contador con enable, y de su *test-bench* , para modelar y simular un contador descendente de 3 bits, con habilitación y reset síncrono activos a nivel alto y entrada de reset asíncrono activa a nivel bajo. El nombre del modelo debe ser *cont3b_en_rst* y el del TB *cont3b_en_rst_tb* –ponga este mismo nombre a los ficheros que los contienen.

Observe que es posible que tenga que detallar alguna prueba adicional para el test del modelo.

No debería invertir más de 10 minutos en la realización de este ejercicio.

PARTE II. Contadores módulo M

Realice las siguientes operaciones:

1. Abra en edición el fichero *cont_mod6_rst.vhd*.

Este fichero contiene el modelo de un contador ascendente de módulo 6, con reset síncrono activo a nivel alto y reset asíncrono activo a nivel bajo. El contador debe seguir una secuencia de cuenta 0, 1, 2, 3, 4, 5, 0, 1…, es decir, la cuenta se incrementa en todos los estados para calcular el siguiente, excepto en el estado de cuenta 5, al que sigue el 0 en lugar del 6. La especificación de este modo de funcionamiento se consigue con una ligera modificación del código que se usa para modelar contadores de módulo máximo (figura 2).

```
process(clk, nRST)
begin
  if nRST = '0' then
    Q <= (others => '0');

  elsif clk'event and clk = '1' then
    if rst = '1' then
      Q <= (others => '0');

    elsif Q = 5 then
      Q <= (others => '0');

    else
      Q <= Q + 1;

    end if;
  end if;
end process;
```

Figura 2.- Contador módulo 6

La condición de cuenta supone la ocurrencia de un flanco de subida de reloj y que el reset síncrono esté inactivo (a ‘0’); en este caso el código evalúa cuál es el estado de cuenta: si el estado de cuenta es 5, se va al estado de cuenta 0 y, si no, se incrementa la cuenta.

```
elsif Q = 5 then
  Q <= (others => '0');

else
  Q <= Q + 1;
```

En este código es importante que observe que la comparación del estado de cuenta (Q = 5) supone la evaluación de la igualdad entre un objeto de tipo *std_logic_vector* (un array de '0's y '1's) y una constante de tipo entero. El uso de esta operación de comparación es posible porque está definida en el paquete *std_logic_unsigned*. En este paquete se definen las siguientes operaciones de comparación de magnitud:

- Igual ("=") y distinto ("/=")
- Mayor que (">") y mayor o igual que (">=")
- Menor que ("<") y menor o igual que ("<=")

Todas estas operaciones están definidas cuando los tipos de los operandos son:

- Uno de tipo entero y el otro de tipo *std_logic_vector*.
- Los dos de tipo *std_logic_vector*

Y el resultado es un valor booleano TRUE cuando se verifica la comparación y False cuando no.

Ejemplos:

En los siguientes ejemplos todas las variables son de tipo *std_logic_vector*.

- B igual a 17
  - B = 17
  - 17 = B
  - B = "10001"

- A menor o igual que B
  - A <= B
  - B > A

- 12 mayor o igual que A
  - 12 >= A
  - A < 12
  - "1100" >= A

- A distinto de 3
  - A /= 3
  - "00000011" /= A

El TB del contador módulo 6 se encuentra en el fichero *cont_mod6_rst_tb.vhd*.

Realice las siguientes operaciones:

1. Abra en edición el fichero *cont_mod6_rst _tb.vhd*.

2. Revise el proceso que codifica las asignaciones a los estímulos síncronos.

3. Compile el modelo del contador y el TB y ejecute una simulación con una duración de 2 microsegundos.

4. Verifique los resultados de la simulación para comprobar el correcto funcionamiento del modelo.

Ejercicio

Reutilice el código del modelo del contador módulo 6 y de su *test-bench* para modelar y simular un contador ascendente de 3 bits y módulo 5 que cuente desde 2 hasta 6. El contador debe disponer de una entrada de reset asíncrono, activa a nivel bajo, que lo inicialice en el estado de cuenta 2, y de una entrada síncrona (*set_to*) cuya activación, a nivel alto, provoque el paso al estado de cuenta 4. El nombre del modelo debe ser *cont2a6_set_to* y el del TB *cont2a6_set_to_tb* –ponga este mismo nombre a los ficheros que los contienen.

No debería invertir más de 15 minutos en la realización de este ejercicio.